Chapter Six: SQL Basic Queries

6.1 Chapter objectives

6.2 Introduction

One powerful statement in SQL is the select statement. It's used to retrieve and display data from one or more tables. Its general form is as follows;

```
SELECT  [DISTINCT/ALL]{*1[column_expression[AS  new
name]][……….]}
FROM       table-name [alias][…….]
[Where           condition]
[Group by Columnlist][Having  condition]
[Order by Columnlist]
```

Where

- *Column expression* signifies the names of a column/expression
- *Table-name* signifies the name of an existing database table/view which you would like to retrieve information from.
- *Alias* is an optional abbreviation for a table/view name.

SQL usually executes the clauses using the following sequence;

FROM	specifies tables to be used.
WHERE	filters the rows subject to some condition
GROUP BY	Forms groups of rows with the same column value
HAVING	filters rows subject to some condition
SELECT	specifies the columns/fields to appear in the output
ORDER BY	specifies the order of the output

- The order of these SQL clauses cannot be charged
- The only mandatory clauses are SELECT and FROM
- The output of select query is another table

6.3 Select statement in SQL

The most commonly used SQL command is SELECT statement. Its role is to query the database and retrieve selected data that follow the conditions we want. In short, the clause is used to query or retrieve data from a table in the database. Its syntax is as follows

```
SELECT column1, column2, ...,columnn
FROM table_name1,table_name2….., table-n;
```

Here, `column1, column2, ... columnn` are the field names of the table you want to select data from while `table_name1,table_name2…. Table_n.`

If you want to select all the fields available in the table, use the following syntax

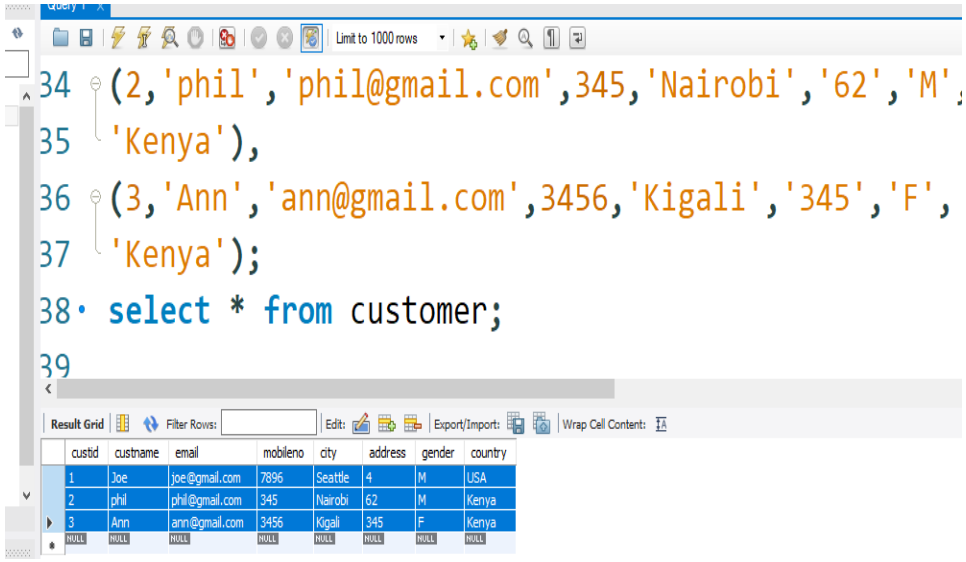
```
SELECT * FROM table_name;
```

To demonstrate use of select clause in SQL, lets populate the table customer with three tuples by executing the following SQL statement

```
insert into customer(custid,custname,email,mobileno,city,address,gender,country)
values(1,'Joe','joe@gmail.com',7896,'Seattle','4','M',
'USA'),
(2,'phil','phil@gmail.com',345,'Nairobi','62','M',
'Kenya'),
(3,'Ann','ann@gmail.com',3456,'Kigali','345','F',
'Kenya');
```

Now, lets retrieve the data from the table customer by using the `select * FROM customer ;`

The effect shall be the following on the screen

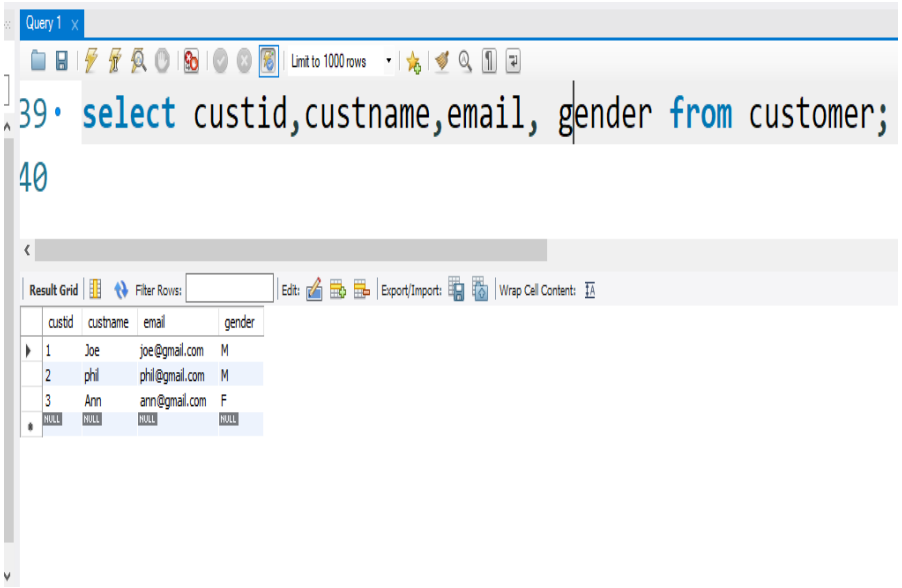


We can also be interested with specific column name from a table.

To do this, we specifying the columns in the SELECT clause using the following syntax

```
SELECT  COLUMN 1, COLUMN 2 ……
FROM        table_name
```

Lets assume we want to retrieve customer id, name, email and gender, we shall execute the following SQL statement as shown on the figure. The output is also presented in the same figure



We can also use clauses like WHERE, GROUP BY, HAVING, ORDER BY with SELECT statement. Its worthy noting that only SELECT and FROM statements are necessary in SQL SELECT statements. Other clauses like WHERE, GROUP BY, ORDER BY, HAVING may be optional.

6.4 Use of distinct key word in SQL

This command is used with SELECT key word to retrieve only distinct or unique data.

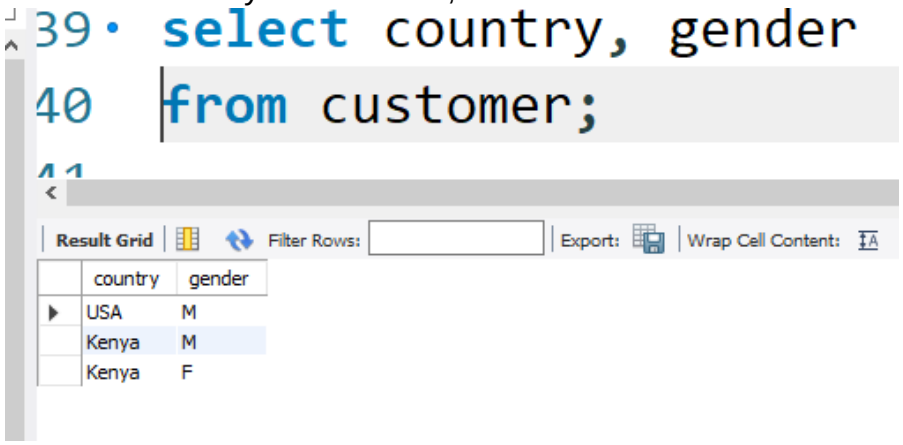
In a table, there may be a chance to exist a duplicate value and sometimes we want to retrieve only unique values. In such scenarios, SQL SELECT DISTINCT statement is used.

Its syntax is as follows

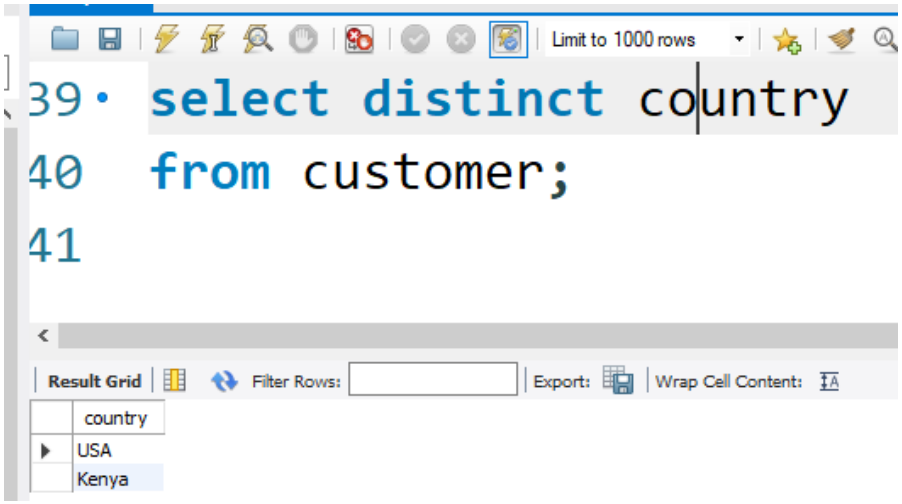
SELECT DISTINCT column_name ,column_name
FROM table_name;

Lets run the following SQL command and observer the output.

Select distinct city from student;



We shall now apply distinct on the country column and observe the output as well



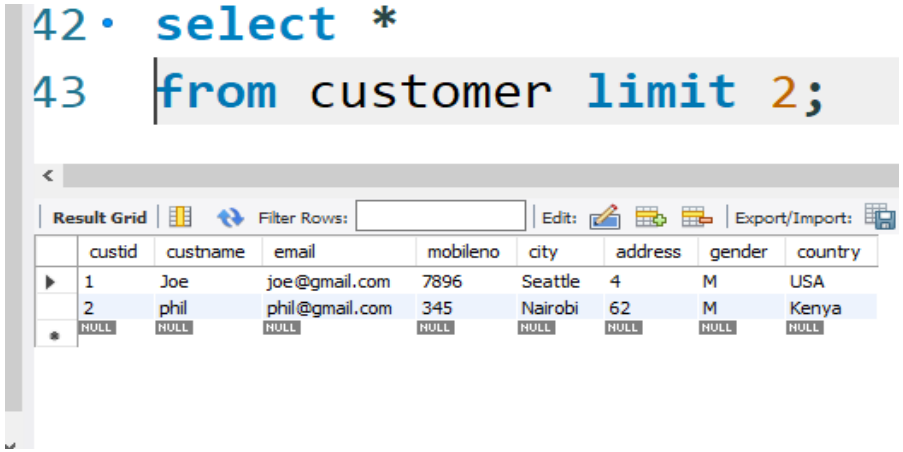
6.5 SQL SELECT TOP

The clause is used to specify the number of records to return and its useful on large tables with thousands of records since returning a large number of records can impact performance.

```
SELECT column_name(s) FROM table_name

WHERE condition LIMIT number;
```

Lets write MySQL to return the first two records in customer table



6.6 Calculated fields in SQL

Some fields may be not stored physically in the database but can be derived when interrogating the data base. E.g. we may want to get the age of our customers by subtracting the current year from the date they were born. SQL has an inbuilt function known as Curdate () which returns the current date. The date will be in form of YYYY-MM-DD. To extract the year part, we can use SQL Year () as shown in the following SQL statement. Since don't have date of birth in our customer table, we shall add a new field known as dob the update the date of birth for our three customers as shown in the following image

```
alter table customer add column dob date;
update customer set dob="2012-05-12" where custid=1;
update customer set dob="2000-12-25" where custid=2;
update customer set dob="1989-02-06" where custid=3;
select * from customer;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

custid	custname	email	mobileno	city	address	gender	country	dob
1	Joe	joe@gmail.com	7896	Seattle	4	M	USA	2012-05-12
2	phil	phil@gmail.com	345	Nairobi	62	M	Kenya	2000-12-25
3	Ann	ann@gmail.com	3456	Kigali	345	F	Kenya	1989-02-06
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

customer 14

Once we execute the following SQL statement, we shall get the following output

```
select custid,custname,year(curdate())-year(dob) as "Age"
from customer;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

custid	custname	Age
1	Joe	8
2	phil	20
3	Ann	31

6.7 Where Clause in SQL

We use the WHERE clause to restrict/filter the tuples/rows that we retrieve in a select statement.

The clause is usually followed by a search condition which specifies to be retrieved. In other words, its used to extract only those records that fulfill a specified condition.

Its syntax is as follows

SELECT column1, column2, ... FROM table_name WHERE condition;

Note: The WHERE clause is not only used in SELECT statement, it is also used in UPDATE, DELETE statement, etc

We have five basic search conditions as defined by the ISO standards.

- `Comparison` which compares the value of one expression to the value of another expression.
- `Range` checks whether the value of a given expression falls within a specified range of values.
- `Set membership` – tests whether the value of an expression equals to one of a set value.
- `Pattern match` – tests if a string matches a specified pattern.
- `Null` – tests whether a column has a null value.

6.7.1 Comparison search condition

```
SELECT    *
FROM      customer
WHERE     country = 'Kenya';
```

Here the table is customer and predicate is country="Kenya". This statement creates a new table containing only those customers whose are from Kenya only

We can also use logical operators such as *AND, OR, AD, NOT* to generate complex predicates and parentheses to show the order of evaluation. Parentheses can help to remove ambiguities in expression as shown in the following example

```
SELECT * FROM customer
WHERE city = 'Nairobi' or country = 'USA';
```

This example return all customer whose city is Nairobi or country is USA or both (from USA and their city is Nairobi) .

6.7.2 Range search condition (between/not between)

The following SQL statement return all customers who were born between 2000-05-12 and up to date

```
51• SELECT * from customer
52  where dob between "2000-05-12" AND Curdate( );
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

custid	custname	email	mobileno	city	address	gender	country	dob
1	Joe	joe@gmail.com	7896	Seattle	4	M	USA	2012-05-12
2	phil	phil@gmail.com	345	Nairobi	62	M	Kenya	2000-12-25
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Assume we want to return all the orders done customers whose quantity is between 1 and 5. We can write the following SQL statement

```
select * from orders
where quantityordered between 1 and 5;
```

Between test will include the end points of the range i.e. if there is an order whose quantity is 1 and 5, they will be included in the result table. The same statement can also be rewritten using logical operators

```
Select * from orders
Where quantityordered > = 1 and quantityordered < = 5;
```

The not between is the negated version of the range test. It will check for values outside the range.

6.7.3 Set membership search condition (IN/NOT IN)

Write an SQL statement that returns all the customers from USA

```
select * from customer
 where country in('USA');
```

write an SQL statement that returns all the customers from Seattle and Nairobi city

```
select * from customer
 where city in ("seattle","Nairobi");
```

The set membership test (IN) will test if a data value matches one/all of a list values.

The negative version (NOT IN) is used to check for data values that do not lie in a specific list of values. For example, write an SQL statement that returns all customers who are not from Kenya.

```
select * from customer
 where country not in ("kenya");
```

note that the statement can also be written as follows

```
Select * from customer where city="Seattle" OR city="Nairobi";
```

However, the IN test provides a more efficient way of expressing search condition especially if the set contains many values.

6.7.4 Pattern match search condition (like/not like)

We can test if a string matches a specified pattern. SQL has two special pattern – matching symbols.

i. % percent character – represents any sequence of zero or more characters.

ii. ii. _Underscore characters: - represents any single character.

All the other characters found in the pattern recognition represent themselves.

E.g.

- City like 'N%' means the first characters must be N but the rest of the string can be anything.
- City like 'N……' mean there must be exactly five characters in the string with the first being an 'N'.
- City like 1% N means an sequence of characters of length alset / with the last character being an 'N'.
- City like '1% Nairobi%' means a sequence of characters of length of any length containing Nairobi.

- City NOT LIKE 'N%' means the first character cannot be 'N'

NB: if the search string can include the pattern – matching character itself we use the key word escape character to represent the pattern – matching character e.g. to check for the string "15%", we use the predicate.

LIKE '15#% ESCAPE '#';

E.g.

```
Select *
FROM customer
Where city LIKE '%Nairobi %';
```

Returns all customer whose city contains Nairobi and can have sequence of characters before and after Nairobi.

6.7.5 NULL search condition (is NULL/IS NOT NULL)

When we use the SQL keyword is NULL to determine if an attribute has a null value. A good example is presented below:

```
SELECT * FROM product where quantity is  NULL;
```

This statement returns tuples whose quantity has not be supplied/has a NULL value.

We can use the key word IS NOT NULL to negate the expressions.

```
SELECT * FROM product where quantity is NOT  NULL;
```

This statement returns tuples whose quantity has been supplied/ doesn't contain a NULL value.

6.8 Order by clause

We can use the ORDER by clause to sort results of a query. The clause allows retrieved rows to be ordered in ascending (ASC) or descending (DESC) order on any column or combination of columns. ASC is the default setting. Its syntax is as follows

```
SELECT column1, column2, ...
FROM table_name
ORDER BY column1, column2, ... ASC|DESC;
```

Example: Write an SQL statement that returns all the customers with their names arranged in an ascending manner

```
Select * from customers
ORDER BY custname ASC;
```

Assuming we would like to order more than one columns e.g. customer name and city, we can use the following SQL statement

```
Select * from customers
ORDER BY custname, city
```

This means that it orders by custname, but if some rows have the same name, it orders them by city

6.9 SQL aggregate functions

They are used to perform some form of summation or aggregation of data. We have five aggregate functions as defined in SQL standards.

i. Count – returns the number of values in a specified column
ii. Sum – returns the sum of values in a specified columns
iii. AVG – returns the average values in a specified columns
iv. Min – returns the minimum value in a specified column
v. Max – returns the maximum values in a specified columns.

The functions operate on a single column and returns a single value. COUNT( ), MAX( ) and MIN ( )operate on numeric and non – numeric data types. SUM ( ) and AVG( ) can only be used on numeric fields only. Apart from the count (*) function, all the other SQL function usually eliminates nulls or duplicate values. Count (*) is a specified use of COUNT which usually count all the rows in a table.

If we need to eliminate duplicates before the function is applied, we can use the key word DISTINCT before the attribute name in the function. It's worth noting that the DISTINCT has no effect on max and min function, but it can have an effect on SUM or AVG functions.

Note

i. SQL aggregate functions can only be used in the SELECT statement or HAVING CLAUSE only and it will be illegal to use it somewhere else.
ii. If the SELECT list includes an aggregate function and no GROUP BY clause is specified to group data together, then no item in the SELECT list can include any reference to a column unless it is an argument of the aggregate function. E.g.

```
SELECT custname, COUNT (*) FROM customer;
```

This is an illegal statement because the query doesn't have a GROUP BY CLAUSE and the column custname in the

SELECT list is used outside an aggregate function.

6.9.1 COUNT () aggregate function

```
SELECT COUNT(column_name)
FROM table_name
WHERE condition;
```

The following statement returns the total number of rows in customer table. The statement also creates a column named My Row.

```
SELECT COUNT (*) As "My Row" FROM customer;
```

6.9.2 Use of COUNT and SUM

The general syntax of the SUM( ) is as follows

```
SELECT SUM(column_name)
FROM table_name
WHERE condition;
```

Find the total number of products and the sum of their price for all products whose price is more than 1000

```
SELECT COUNT (*) AS "Total student", sum (price) as "Total Sum" FROM product
WHERE (price) >1000;
```

6.9.3 Use of MIN, MAX, AVG

The syntax for MIN( ) is as follows

```
SELECT MIN(column_name)
FROM table_name
WHERE condition;
```

Write SQL statement which return the minimum price of products we have in the database

```
SELECT MIN(price) as "Minimum Price" FROM products
```

The syntax for MAX( ) is as follows

```
SELECT MAX(column_name)
FROM table_name
WHERE condition;
```

Write SQL statement which return the maximum price of products we have in the database

```
SELECT MAX(price) as "Maximum Price" FROM products
```

The syntax for AVG( ) is as follows

```
SELECT AVG(column_name)
FROM table_name
WHERE condition;
```

Write SQL statement which return the average price for all products we have in the database

```
SELECT AVG(price) as "Average Price" FROM products
```

You can find more functions from this link

https://www.w3schools.com/sql/sql_ref_sqlserver.asp

6.10 Grouping results (group by clause)

Group by clause is used in SQL to group attributes together. A query that includes a GROUP BY CLAUSE is called a grouped query since it groups the data from the SELECT tables and produces a single summary for each group. The attribute named in the GROUP BY clause are referred to as grouping columns.

i. When GROUP BY is used, each item in the SELECT list must be single – value per group.
ii. The clause may contain only; column names or aggregate functions
iii. not all the columns that are present in the select clause can be included in the GROUP BY
iv. Any column that is present in the GROUP BY must be present in the SELECT list.
v. If a WHERE CLAUSE is used, the WHERE clause is first applied then groups are formed from remaining triples that meet the search condition.

Example of GROUP BY clause

```
SELECT gender, count (custid) as "Total customers"
  FROM customer
   GROUP BY GENDER
   ORDER BY gender;
```

It's not necessary to include the attribute custid in the GROUP BY CLAUSE since it appears only in the SELECT list in an aggregate function (count)

However, since gender attribute is not part of an aggregate function, we must include in the GROUP BY list as SQL syntax demands. The result will be as follows

Result

Gender	Total Customer
M	2
F	1

6.11 Views in database

They are a dynamic result of one or more relational operations on base relations to produce another relation. It is a

virtual/derived relation that does not necessary exists in the database but can be produced upon request by a particular user at a time of request.

Contents of a view can be derived as query on one or more relations. Views are dynamic in that changes done on the base relation that affect the view are reflected immediately.

You can add SQL functions, WHERE, and JOIN statements to a view and present the data as if the data were coming from one single table.

6.11.1 Purpose of views

- Permits users to access data in a way that customized to meet their needs so that the same data is seen by different users differently at the same time.
- Provides powerful and flexible security of hiding parts of the database from users since the users are not aware of the existence of any attributes/tuples that are missing from their view
- Helps to simplify complex operations on the base relations. It's always the work of the DBMS to perform translation into equivalent operations.

6.11.2 Syntax for creating views in SQL

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
```

Note: A view always shows up-to-date data! The database engine recreates the data, using the view's SQL statement, every time a user queries a view.

Example 1: Create a view to list all customers from *USA* and are *Males*.

```
CREATE VIEW USACUSTOMERS  AS
SELECT *
FROM customers
Where country='USA' AND gender='M';
```

Example 2: Create a view to list all orders done by a customer who has customer ID 1 and the order was done on "2012-06-28"

```
CREATE VIEW customerOrder  AS
SELECT * FROM orders
WHERE custid=1 AND orderdate="2012-06-28";
```

Example 3: Create a view to list named customerOrders which returns the orders done by customers between 2018-02-28 and current date. The view gets its information from more than one table (customer, orders and products)

```
CREATE VIEW customerOrders AS
  SELECT
    c.custid,
    c.custname,
    p.prodid,
    p.prodname,
    p.price,
    o.orderid,
    o.orderdate,
    o.quantityordered
  FROM
    orders o,
    customer c,
    product p
  WHERE
    c.custid = o.custid
      AND p.prodID = o.prodID
      AND o.orderdate BETWEEN '2018-02-28' AND CURDATE();
```

6.11.3 Calling/invoking views in SQL

The following syntax is used

*SELECT * FROM view-name;*

For example to call the above view into execution, we can use the following statement

```
SELECT * customerOrders;
```

The result shall be execution of the SQL statements with the AS block resulting into a query table containing columns specified in the SELECT clause which meets the specified condition

6.11.4 SQL Updating a View

A view can be updated with the CREATE OR REPLACE VIEW command.

SQL CREATE OR REPLACE VIEW Syntax

```
CREATE OR REPLACE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

For example, the following SQL statement removes the column/field adds the "custname" column to the "CustomerOrders" view

```
CREATE OR replace VIEW customerOrders AS
  SELECT
    c.custid,

    p.prodid,
    p.prodname,
    p.price,
    o.orderid,
    o.orderdate,
    o.quantityordered
  FROM
    orders o,
    customer c,
    product p
  WHERE
    c.custid = o.custid
      AND p.prodID = o.prodID
      AND o.orderdate BETWEEN '2018-02-28' AND CURDATE();
```

6.11.5 DROPPING a view

A view is deleted with the DROP VIEW command which as the syntax `SQL DROP VIEW`

The following SQL drops the "customerOrders" view

```
DROP VIEW customerOrders;
```

6.12 Multiple Table Queries

Single table queries have a major drawback; we only get information from a single table. In order to combine columns from several tables, we need to use the join operation. This

operation combines information from two or more tables by forming pairs of related rows. The join operation is usually achieved by including two or more tables in the SELECT statement using a comma as a separate and using the WHERE clause to specify the join group attribute(s). We can also include an alias for a table which is separated from table by a space as shown below

```
SELECT
    O.orderID,
    O.rderdate,
    O.Quantity,
    Ordered,
    p.prodID,
    p.prodPrice,
    p.prodName
FROM
    product p,
    Orders O
WHERE
    p.prodid = o.prodid;
```

In the above example p and O are known alias in SQL and they are used to qualify column name in case there is an ambiguity of the column source. They are also used to substitute the name of a table in an SQL query.

Since we would like display the details from products and orders table, we need to join the two tables. The SELECT clauses list all the columns to be displayed. We use the alias *p* to reference to all columns from product table. The alias o will be used to reference to all columns from the orders table. Through the aliases, we can solve the fields ambiguity

In order to return the required rows we include these rows from both tables that have identical values using the search condition `( p.prodid = o.prodid).`

The same case will also apply if we wish to join more than two tables. We refer our earlier example

```
SELECT
    c.custid,
    p.prodid,
    p.prodname,
    p.price,
    o.orderid,
```

```
    o.orderdate,
    o.quantityordered
  FROM
    orders o,
    customer c,
    product p
  WHERE
    c.custid = o.custid
      AND p.prodID = o.prodID
      AND o.orderdate BETWEEN '2018-02-28' AND
curdate()
```

In our new example, we wish to get data from three tables; products, customer and orders and therefore we shall join the three tables. The SELECT clauses list all the columns to be displayed. We use the alias p to reference to all columns from product table, *o* to reference to all columns from the orders table and *c* to reference to columns coming from customer table . Through the aliases, we can solve the fields ambiguity

In order to return the required rows we include the SQL join `c.custid = o.custid` `AND p.prodID = o.prodID`

We have added another condition, *o.orderdate BETWEEN '2018-02-28' AND curdate ( )* Which further filters the rows being return so that the order date of the orders is between today and "2018-02-28".

The topic SQL joins will be delved further in another module at degree level , CMT 302:-Advanced Database Systems

6.13 Chapter summary

6.14 Chapter exercise

6.15 Further reading suggestions

https://www.w3schools.com/sql/

https://www.javatpoint.com/mysql-tutorial